ПРИКЛАДНАЯ ЛИНГВИСТИКА

ББК Ш141.2-2 + Ш103.2

ОБ УНИВЕРСАЛЬНОЙ ПЛАТФОРМЕ ЛИНГВИСТИЧЕСКОГО ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

С.О. Шереметьева, Н.Ю. Дюмин

Описана модель платформы разработки и функционирования приложений автоматической обработки текстов на естественном языке: приведено общее описание системы, а также описание составляющих его компонентов.

Ключевые слова: компьютерная лингвистика, прикладная лингвистика, автоматический перевод, машинный перевод, автоматическая обработка текста, разработка приложений, программирование, инструментарий разработчика.

1. Введение

Несмотря на разнообразие подходов и более чем шестидесятилетние исследования (их началом можно считать время возникновения машинного перевода), экономически необходимые высококачественные системы автоматической обработки естественного языка: автоматические перевод, информационный поиск, реферирование и аннотирование и т.д., так и не стали реальностью. Естественный язык настолько необъятен и неоднозначен, что создание компьютерных систем, способных оперировать всем многообразием лингвистических знаний, требует бесконечно много усилий, времени и финансирования.

Основные тенденции в области оптимизации создания лингвистических компьютерных систем включают в себя разработку стратегий повторного использования и интеграции[1] уже существующего в новые разработки, а также создание инструментария широкого диапазона, который мог бы быть адаптирован для различных целей. Такой инструментарий ориентирован как на оптимизацию создания лингвистического обеспечения систем, например, лексиконов, грамматик правил и алгоритмов оперирования ими, так и на усовершенствование и ускорение разработки лингвистического программного обеспечения. Чаще всего – это программы, допускающие обновление и компиляцию лингвистических знаний на одном или нескольких естественных языках, как, например, программа BOAS, представляющая собой инструментарий сбора грамматических знаний для языков с отсутствием электронных ресурсов в целях быстрой разработки систем автоматического перевода[2].

Все большую популярность приобретает модульная архитектура систем обработки естественного языка, в которых модули, выполняющие отдельные функции (например, морфологический/синтаксический анализ или синтез) могут комбинироваться в новых одноязычных или многоязычных системах, как, например, при добавлении новых языковых пар в системе автоматического перевода SYSTRAN[3] или в системах автоматического перевода и синтеза патентов АвтоТранс и АвтоПат[4].

Тем не менее, проблема создания инструментария, специально ориентированного на программирование лингвистических систем, который, с одной стороны, позволял бы учитывать специфику лингвистических приложений, а с другой, обеспечил бы совместимость программного обеспечения, созданного различными разработчиками для отдельных типов автоматической обработки естественного языка, так чтобы вновь создаваемые системы могли бы включать в себя ранее созданные программные модули, до сих пор не решена.

В настоящее время многие лингвистические системы имплементируются на разных языках программирования для разных платформ и, таким образом, заведомо несовместимы. При этом, несмотря на то, что среди существующих языков программирования можно выделить ряд более приспособленных к решению лингвистического задач, а также ряд языков, возможности работы с естественными языками которых значительно расширены дополнительными библиотеками или наборами классов, это не делает их в полной мере ориентированными на естественный язык, что ограничивает эффективность и возможности разработчиков.

Практически каждый раз, с началом нового проекта исследовательская группа чаще всего начинает с нуля (исключение составляют группы проектов, которые подобно указанным выше, изначально созданы для функционирования во взаимодействии) приходится выбирать платформу, тем самым ограничивая круг пользователей, язык программирования, а также создавать модель взаимодействия его структур со структурами естественного языка. При этом создание сложных приложений высокого уровня, таких как, например, системы автоматического перевода, в большинстве случаев, подразумевает создание собственных моду-

лей низкого уровня, например, модулей морфологического и синтаксического анализа.

Опыт исследований в области автоматической обработки естественного языка, имеющих положительные результаты, позволяет утверждать, что экономия, а в идеальном случае, минимизация усилий разработчиков при повышении качества и снижении стоимости лингвистических программ зависит от решения двух взаимосвязанных проблем:

а) проблемы отбора и организации лингвистических знаний (например, лексиконов, грамматик и алгоритмов оперирования ими)

б) проблемы создания лингвистического программистского инструментария, который, с одной стороны, учитывал бы специфику лингвистических приложений и его пользователей, вычислительных лингвистов без фундаментальных навыков программирования, а с другой, обеспечивал бы совместимость программного обеспечения, созданного различными разработчиками.

В настоящей статье предпринимается попытка внести определённый вклад в решение второй из указанных выше проблем путем создания универсальной платформы разработки и функционирования лингвистического программного обеспечения (ЛПО).

Статья организована следующим образом: Прежде всего, мы даём обзор наиболее значимых проектов в области ЛПО: ЭТАП[5], GATE[6] и Ellogon[7], затем приводим спецификацию предлагаемой нами универсальной платформы ЛПО, вслед за которой описываем предлагаемую реализацию нашей платформы, язык программирования и предварительный эксперимент, за которым следует заключение с выводом о перспективах нашего подхода.

2. Проекты ЭТАП, GATE и Ellogon

Наиболее интересными и значимыми для нашей работы проектами в области ЛПО, являются система ЭТАП, GATE и Ellogon. Система ЭТАП, разрабатывается в Лаборатории компьютерной лингвистики Института проблем передачи информации РАН под руководством профессора Богуславского И.М. ЭТАП – это компьютерная версия модели, имитирующая языковое поведение человека, т.е. его способность производить тексты на естественном языке и понимать их. Система имеет вид полифункционального многоязычного процессора и состоит из набора модулей, которые обеспечивают функционирование ряда построенных в Лаборатории систем переработки текстов, таких, как машинный перевод, порождение русских текстов по смысловому заданию на базе Универсального сетевого языка (UNL)[8], перефразирование предложений на данном естественном языке и т. п. Эта работа ведется в рамках программы[9], разрабатываемой большим международным консорциумом под эгидой ООН, и ставит своей целью обеспечить для пользователей Интернета возможность получать и распространять информацию на родном языке.

Система GATE, разработка которой ведется в университете города Шеффилд (Англия) под руководством профессора Х. Канингема, представляет собой платформу разработки лингвистических приложений, основанную на том, что любое приложение может быть разбито на компоненты, каждый из которых может быть использован в других приложениях, созданных в этой же среде. GATE уже содержит в себе некую модель представления текста, основываясь на которой разработчик может создавать приложения используя элементы текста, как естественно-языковые структуры. Разработка компонентов в GATE ведется на языке программирования Java, который, изначально не был создан для работы с естественно-языковыми структурами но, тем не менее, имеет ряд преимуществ, основным из которых является возможность кроссплатформенного использования созданных приложений.

Другой интересный проект – Ellogon, разработка которого ведется в лаборатории при Институте информатики и телекоммуникаций национального центра научных исследований «Демокритос» (Греция). Система Ellogon также представляет собой платформу разработки и функционирования ЛПО с модульной архитектурой систем. Особенностью этой системы является возможность разработки компонентов на нескольких языках программирования (Java, C++, TCL, Perl и Python) Еще одна особенность системы – возможность создания самостоятельных исполняемых файлов. Несмотря на всю гибкость систем, они не предлагают своего языка программирования ориентированного на разработку ЛПО, а, значит, не препятствуют появлению избыточных с точки зрения разработчика ЛПО конструкций. Таким образом, разработчик ЛПО для этих платформ должен обладать отличным знанием одного из перечисленных языков программирования, что не всегда реально даже для специалистов в области компьютерной лингвистики.

3. Спецификация универсальной платформы ЛПО

Под платформой разработки и функционирования ЛПО мы понимаем набор инструментальных средств, обеспечивающих возможность создания особого класса программного обеспечения для автоматизации различных видов человеческой деятельности, предполагающих обработку естественного языка. Классификация ЛПО предложена нами ранее[10].

Анализ современных работ и собственные исследования авторов позволяет нам сформулировать основные характеристики универсальной платформы ЛПО следующим образом:

- основой системы должен быть язык программирования высокого уровня, имеющий естественно-языковую ориентацию, т.е. изначально раз-

работанный для манипулирования структурами естественного языка и не требующий фундаментальных знаний в области программирования;

• система должна работать по возможности во всех наиболее распространенных средах, а значит, должна представлять собой виртуальную машину – процессор, управляемый командами вышеуказанного языка;

• приложения, функционирующие в системе должны представлять собой набор взаимодействующих модулей, предоставляя возможность использования одной системой модулей других;

• система должна позволять создавать и выполнять приложения всех степеней открытости (см. классификацию по степени открытости системы для конечного пользователя);

• система должна позволять выстраивать модули в организации иерархически разного уровня;

• система должна позволять осуществлять взаимодействие с базами данных, с файловыми системами и с конечным пользователем посредством графического интерфейса пользователя;

• система должна предпочтительно использовать таблицу кодировки Unicode, поскольку для языкового материала важна возможность обработки специальных символов;

• система должна позволять централизованную обратную связь с создателем модуля и предусмотреть возможность распределенного автоматизированного создания документации к проекту.

4. Реализация платформы

Наиболее подходящей нам представляется реализация платформы в виде виртуальной машины (ВМ), состоящей из трех компонентов:

• процессор, исполняющий набор элементарных команд;

• компилятор, преобразующий сценарии, написанные на языке программирования, принятом в системе, в команды процессора виртуальной машины;

• набор шлюзов, осуществляющих связь ВМ с внешней системой: с базами данных, с файловой системой, с графической системой (рис. 1).

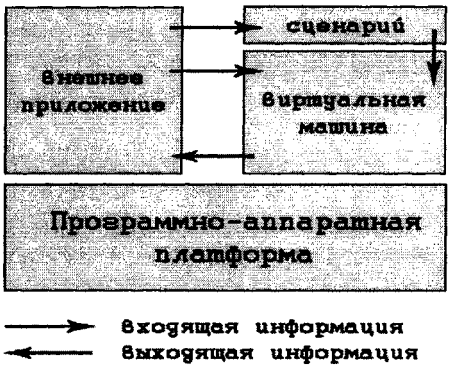

Рис. 1. Принцип взаимодействия приложений с виртуальной машиной

Внешнее приложение вызывает выполнение заданного сценария на виртуальной машине с передаваемыми параметрами, виртуальная машина, в свою очередь, возвращает результат. Внешнее приложение, как и виртуальная машина, платформозависима. Сценарий платформонезависим. В виртуальной машине заложен базовый набор функций ввода-вывода информации, позволяющий создавать элементарные псевдовнешние приложения для манипулирования сценариями. Поток информации, изображенный в виде стрелок, осуществляется с помощью третьего компонента ВМ. Работа первых двух описана на нижеследующем рисунке (рис. 2):

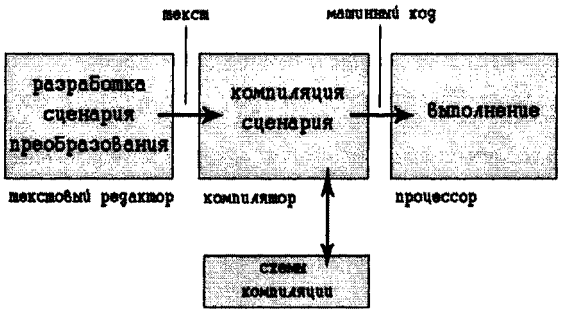

Рис. 2. Принцип разработки и функционирования сценариев в виртуальной машине

Разработка сценариев на предложенном языке программирования ведется в текстовом редакторе. В дальнейшем планируется создание интегрированной среды разработки приложений для еще большего упрощения и большей автоматизации процесса разработки. После чего текст поступает в компилятор, преобразуется в код процессора виртуальной машины при помощи схем преобразования, часть из которых может быть определена пользователем. На последнем этапе поступившая информация проходит трансформации по скомпилированному сценарию, и виртуальная машина возвращает результат.

Наша модель предполагает, что каждое приложение, выполняемое на машине, состоит из модулей, производящих операции над входящей информацией, обрабатывающих ее особым образом и выдающих на выходе новую информацию для другого модуля, либо для предъявления пользователю в качестве результата работы приложения.

Сценарии обработки входящей информации внутри модуля, равно как и сопряжение модулей, будут описаны специальным языком программирования, исполняемым на виртуальной машине. Разработчики ЛПО будут иметь возможность определять дополнительные структуры, описывающие логику программы, т.е. вводить конструкции высокого уровня, а также алгоритмы их преобразования в команды лингвистического процессора. Стандартные же, предопределенные структуры должны наилучшим образом описать круг возможных операций, производимых как над симво-

лами, так и над их группами, в том числе словами (здесь «слово» – группа символов, окруженная символами пробела) и массивами слов. Такой подход даст возможность описания максимально большого количества методов автоматической обработки ЕЯ. Здесь мы затронули проблему типов переменных, функционирующих в ВМ.

Мы предполагаем, что на первом этапе реализации системы в переменных как таковых необходимости нет вовсе, кроме двух стандартных – переменной входящей информации и переменной выходящей информации. Структура переменных представляет собой двухмерный массив, ячейки которого разработчик заполняет данными по своему усмотрению. В системе используется один тип данных – строка, в том числе нулевая строка и односимвольная строка. Для осуществления циклов будет введен специальный класс объектов – счетчики.

5. Язык программирования

Предлагаемый нами язык программирования предназначен для лингвистических задач и ориентирован на пользователя, не обладающего высокой программистской квалификацией. Система команд и структур языка составлена наиболее понятным образом, что упрощает процесс его освоения.

Программирование сценариев для виртуальной машины, в частности задача программирования интерфейсов, решается в автоматическом, либо полуавтоматическом режиме, с помощью псевдовнешних приложений – разработчику не придется описывать интерфейс самостоятельно, а значит привлекать знания об устройстве той или иной системы (Windows, X11 и т.д.). Как следствие, при использовании вспомогательных программ-шлюзов, приложения можно будет запускать на любой системе (оконной или консольной: Windows, UNIX, Mac и т.д.).

Компилятор языка будет интерпретировать сценарий в последовательность команд виртуального процессора, архитектура которого будет наиболее подходящей для решения лингвистических задач, например, будут введены новые структурные элементы, а также новые, по сравнению с архитектурой процессоров семейства x86, команды.

Благодаря реализации платформы в виде виртуальной машины с возможностью компиляции кода, а не в виде транслятора, существенно расширяется возможность введения дополнительных структур.

Структуры языка будут обеспечивать стандартное ветвление (IF-ветвление, WHILE-циклы, FOR-циклы, и т.д.), а также специфические операции, например, структура, описывающая процесс выделения подстроки в строке с заданными параметрами и выполнение действий в зависимости от значения подстроки. В качестве дополнительных структур планируется ввести стандартные операции из наиболее распространенных моделей обработки естественного языка.

Существенным достоинством предлагаемого языка программирования является то, что написание программ обработки естественного языка на предложенном нами языке программирования будет более эффективным потому, что специальные лингвистические структуры позволят существенно сократить размер программ.

6. Эксперимент

С целью тестирования целесообразности нашего подхода мы провели эксперимент по созданию программной реализации элементов нашей модели: создали прототип языка программирования, описав базовый набор структур, необходимых для реализации морфологических анализаторов, и интерпретатор данного языка в PHP-код.

В рамках этого предварительного эксперимента мы ограничились стандартными средствами вывода информации, предоставляемыми PHP-интерпретатором, не реализовывали компилятор и процессор, а также компоненты взаимодействия с базами данных, файловыми системами, поскольку целью эксперимента было выявить насколько целесообразно использование предложенного нами языка, как такового.

В качестве примера мы реализовали алгоритм морфологического анализа правильных испанских глаголов в активном залоге[11], написав примерно 140 строк кода. В своей работе Ставцева реализует алгоритм на языке программирования PHP с помощью примерно 1140 строк кода, что свидетельствует о правильности выбранного нами подхода.

7. Заключение

В настоящей работе мы показали, что для оптимизации создания лингвистических компьютерных систем (автоматических перевода, информационного поиска, аннотирования и реферирования и т.д.) к числу важнейших проблем, требующих оперативного решения, относится проблема лингвистического программного обеспечения.

В качестве наиболее эффективного подхода решения этой проблемы нами предлагается:

а) создание универсальной платформы разработки и функционирования лингвистического программного обеспечения, которая позволила бы объединить усилия разработчиков и в значительной мере упростить процесс создания ЛПО за счет использования общих модулей;

б) создание нового языка программирования, ориентированного на разработку ЛПО и не предполагающего фундаментальных знаний в области программирования, что облегчит труд лингвистов-разработчиков.

Дана общая спецификация платформы, формулирующая основные требования к такого рода программным продуктам. Приведены краткие описания реализации платформы и лингвистически ориентированного языка программирования. Описан предварительный эксперимент по апробации реалистичности и целесообразности нашего подхода к универсальному лингвистическому программному

обеспечению. Полученные результаты свидетельствуют о правильности выбранного нами подхода.

[1] Prieto-Diaz R. Status report: software reusability. IEEE Software 10(3), 1993; Thomas I., and Nejmeh B. Definitions of tool integration for environments. IEEE Software. 9(2), 1992.

[2] Sheremetyeva, S and S. Nirenburg Towards A Universal Tool For NLP Resource Acquisition Proceedings of LREC-2000 (Second International Conference on Language Resources and Evaluation) Athens, Greece., 2000.

[3] Surcin S., Lange E. and Senellart J. Rapid Development of New Language Pairs at SYSTRAN. in Proceedings of MT Summit XI, Copenhagen, Denmark, 2007.

[4] Sheremetyeva S. On Portability of Resources for Quick Ramp-Up of Multilingual MT for Patent Claims. in Proceedings of the workshop on Patent Translation in conjunction with MT Summit XI, Copenhagen, Denmark, 2007.

[5] http://cool.iitp.ru/iitp/2002/lab15.pdf

[6] Cunningham, H JAPE: a Java Annotation Pattern Engine. Research Memorandum CS-99-06, Department of Computer Science, University of Sheffield, England, 1999.

[7] Georgios Petasis, Vangelis Karkaletsis, Georgios Paliouras, Ion Androutsopoulos and Constantine D. Spyropoulos Ellogon: A New Text Engineering Platform, 2002.

[8] Saqer Abdel-Rahim, Asad Abu Libdeh, Fakher Sawalha, Mohammed K K. Odeh Universal Networking Language (UNL) a Means to Bridge the Digital Divide. – http://www.worldbank.org/mdf/mdf4/papers/abdelrahim.pdf.

[9] Universal Networking Digital Language Foundation Website. – http://www.undl.org

[10] Дюмин Н.Ю. Моделирование инструментария разработчика систем обработки естественного языка. Классификация систем. Курсовая работа, 2008.

[11] Ставцева И.В. Морфологический анализ глагола испанского языка. Курсовая работа, 2008.

Поступила в редакцию 23 мая 2008 г.